

Stochastic Gradient Descent

Optimization Principles

Hoa T. Vu

Vector Notation

Linear model $\hat{y} = \theta^\top x = \sum_{i=1}^n \theta_i x_i$

Linear model $\hat{y} = \theta^\top x = \sum_{i=1}^n \theta_i x_i$

MSE $\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta^\top x_i)^2$

Linear model $\hat{y} = \theta^\top x = \sum_{i=1}^n \theta_i x_i$

MSE $\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta^\top x_i)^2$

Gradient $\nabla_{\theta} \mathcal{L}(\theta) = \frac{-2}{n} \sum_{i=1}^n (y_i - \theta^\top x_i) x_i$

Motivation

Full Gradient Descent

Each update requires the gradient over *all* n samples:

$$\nabla \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\theta)$$

where $\ell_i(\theta)$ is the loss on the i -th sample alone.

Every parameter update touches every data point.

MSE: Each Term Requires One Sample

Per-sample gradient for linear regression $\hat{y}_i = \theta^\top x_i$:

$$\nabla_{\theta} \ell_i(\theta) = -2 (y_i - \theta^\top x_i) x_i$$

MSE: Each Term Requires One Sample

Per-sample gradient for linear regression $\hat{y}_i = \theta^\top x_i$:

$$\nabla_{\theta} \ell_i(\theta) = -2 (y_i - \theta^\top x_i) x_i$$

The full gradient averages over all n samples:

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{-2}{n} \sum_{i=1}^n (y_i - \theta^\top x_i) x_i$$

Why This Is Expensive

- One gradient step = n individual gradient evaluations
- For $n = 10^6$, that is 10^6 dot products per update
- Modern datasets have $n = 10^8$ or more

Why This Is Expensive

- One gradient step = n individual gradient evaluations
- For $n = 10^6$, that is 10^6 dot products per update
- Modern datasets have $n = 10^8$ or more

Key Question

Can we speed up gradient descent?

Example: Full Gradient for MSE

Model $\hat{y}_i = \theta x_i$, loss $\ell_i(\theta) = (y_i - \theta x_i)^2$, gradient $\nabla_{\theta} \ell_i = -2(y_i - \theta x_i) x_i$.

i	x_i	y_i	$\hat{y}_i$ ($\theta = 2$)	$\nabla_{\theta} \ell_i$
1	1	3	2	$-2(3 - 2)(1) = -2$
2	2	5	4	$-2(5 - 4)(2) = -4$
3	3	8	6	$-2(8 - 6)(3) = -12$
4	4	9	8	$-2(9 - 8)(4) = -8$

Example: Full Gradient for MSE

Model $\hat{y}_i = \theta x_i$, loss $\ell_i(\theta) = (y_i - \theta x_i)^2$, gradient $\nabla_{\theta} \ell_i = -2(y_i - \theta x_i) x_i$.

i	x_i	y_i	$\hat{y}_i$ ($\theta = 2$)	$\nabla_{\theta} \ell_i$
1	1	3	2	$-2(3 - 2)(1) = -2$
2	2	5	4	$-2(5 - 4)(2) = -4$
3	3	8	6	$-2(8 - 6)(3) = -12$
4	4	9	8	$-2(9 - 8)(4) = -8$

$$\nabla \mathcal{L}(\theta) = \frac{1}{4}(-2 - 4 - 12 - 8) = -6.5 \quad \Rightarrow \quad \theta \leftarrow 2 - \alpha \cdot (-6.5)$$

Example: Full Gradient for MSE

Model $\hat{y}_i = \theta x_i$, loss $\ell_i(\theta) = (y_i - \theta x_i)^2$, gradient $\nabla_{\theta} \ell_i = -2(y_i - \theta x_i) x_i$.

i	x_i	y_i	$\hat{y}_i (\theta = 2)$	$\nabla_{\theta} \ell_i$
1	1	3	2	$-2(3 - 2)(1) = -2$
2	2	5	4	$-2(5 - 4)(2) = -4$
3	3	8	6	$-2(8 - 6)(3) = -12$
4	4	9	8	$-2(9 - 8)(4) = -8$

$$\nabla \mathcal{L}(\theta) = \frac{1}{4}(-2 - 4 - 12 - 8) = -6.5 \quad \Rightarrow \quad \theta \leftarrow 2 - \alpha \cdot (-6.5)$$

With $n = 10^6$ rows, every single update requires 10^6 such computations.

SGD

Stochastic Gradient Descent

At each step, pick **one** sample i_t uniformly at random:

$$\mathbf{g}_t = \nabla \ell_{i_t}(\theta_t), \quad \theta_{t+1} = \theta_t - \alpha_t \mathbf{g}_t$$

Cost per step: **one** gradient evaluation instead of n .

SGD: Unbiased Estimate

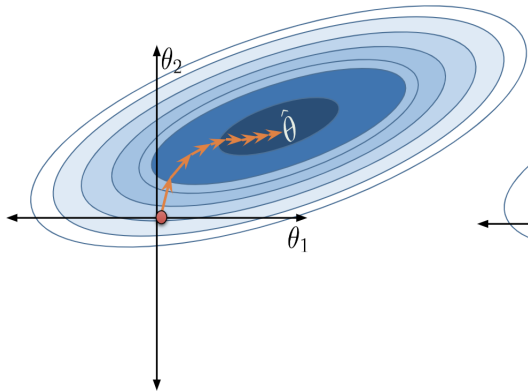
The single-sample gradient g_t is a noisy but **unbiased** estimate:

$$\mathbb{E}[g_t] = \nabla \mathcal{L}(\theta_t)$$

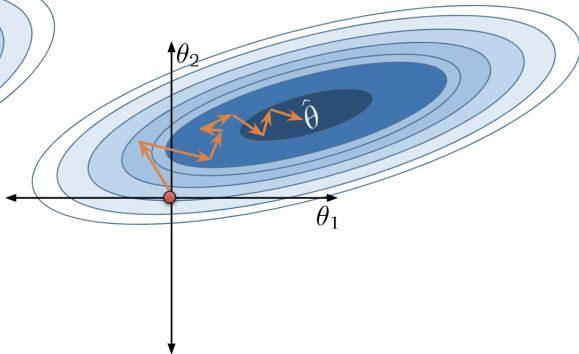
On average, g_t points in the same direction as the true gradient.

Visualization

Batch Gradient Descent



Stochastic Gradient Descent



SGD: Pros and Cons

Pros

- One pass per step (cheap)

Cons

- High variance, erratic path

SGD: Pros and Cons

Pros

- One pass per step (cheap)
- Can escape shallow local minima

Cons

- High variance, erratic path
- Requires decreasing α_t to converge

SGD: Pros and Cons

Pros

- One pass per step (cheap)
- Can escape shallow local minima
- Works in online / streaming settings

Cons

- High variance, erratic path
- Requires decreasing α_t to converge
- Harder to parallelize than mini-batch

Mini-batch SGD

Use a random subset $\mathcal{B}_t \subseteq \{1, \dots, n\}$ of size B :

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \alpha_t g_t$$

Mini-batch SGD

Use a random subset $\mathcal{B}_t \subseteq \{1, \dots, n\}$ of size B :

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \alpha_t g_t$$

	$B = 1$	$1 < B < n$	$B = n$
Name	SGD	Mini-batch SGD	Full GD
Cost/step	cheap	moderate	expensive
Variance	high	medium	none

Mini-batch SGD

Use a random subset $\mathcal{B}_t \subseteq \{1, \dots, n\}$ of size B :

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \alpha_t g_t$$

	$B = 1$	$1 < B < n$	$B = n$
Name	SGD	Mini-batch SGD	Full GD
Cost/step	cheap	moderate	expensive
Variance	high	medium	none

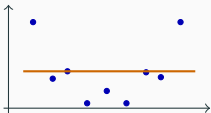
In practice $B \in \{32, 64, 128, 256\}$ balances noise and hardware efficiency.

Bias-Variance Tradeoff

Underfitting, Overfitting, and Just Right

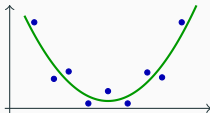
Underfitting

high bias, low variance



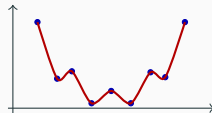
Just Right

low bias, low variance



Overfitting

low bias, high variance



What Are Bias and Variance?

Imagine training on many different datasets drawn from the same distribution.

What Are Bias and Variance?

Imagine training on many different datasets drawn from the same distribution.

Bias how far off the average prediction is from the truth

Variance how much the prediction changes across datasets

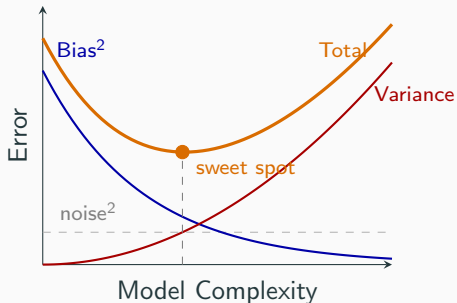
Bias–Variance Tradeoff

$$\mathbb{E}[\text{error}] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \sigma^2$$

Bias–Variance Tradeoff

$$\mathbb{E}[\text{error}] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \sigma^2$$

The **sweet spot** is the minimum of the total error curve.



Regularization

What Is Regularization?

Add a penalty to the loss to discourage large weights:

$$\mathcal{L}_{\text{reg}}(\theta) = \underbrace{\mathcal{L}(\theta)}_{\text{fit the data}} + \lambda \underbrace{R(\theta)}_{\text{penalty}}$$

What Is Regularization?

Add a penalty to the loss to discourage large weights:

$$\mathcal{L}_{\text{reg}}(\theta) = \underbrace{\mathcal{L}(\theta)}_{\text{fit the data}} + \lambda \underbrace{R(\theta)}_{\text{penalty}}$$

L2 (Ridge) $R(\theta) = \|\theta\|_2^2$ $\rightarrow$ shrinks all weights, but never to exactly zero

L1 (Lasso) $R(\theta) = \|\theta\|_1$ $\rightarrow$ drives some weights to **exactly zero**

Subgradient Descent for L1

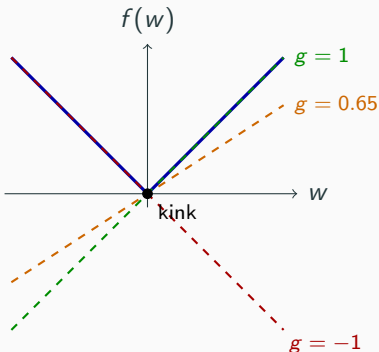
What Is a Subgradient?

At a kink, there is no unique tangent — but a whole **range of valid slopes**. Any slope from that range is a **subgradient**.

What Is a Subgradient?

At a kink, there is no unique tangent — but a whole **range of valid slopes**. Any slope from that range is a **subgradient**.

A slope g is valid if its line lies *below* f everywhere.



Subgradient of $|w|$: Using $\text{sign}(w)$

Gradient descent needs $\frac{d}{dw}|w|$ at every step — but that derivative does not exist at $w = 0$.

Fix: pick any subgradient — the standard choice is $\text{sign}(w)$:

$$\text{sign}(w) = \begin{cases} +1, & w > 0 \\ 0, & w = 0 \\ -1, & w < 0 \end{cases}$$

Subgradient of $|w|$: Using $\text{sign}(w)$

Gradient descent needs $\frac{d}{dw}|w|$ at every step — but that derivative does not exist at $w = 0$.

Fix: pick any subgradient — the standard choice is $\text{sign}(w)$:

$$\text{sign}(w) = \begin{cases} +1, & w > 0 \\ 0, & w = 0 \\ -1, & w < 0 \end{cases}$$

At $w = 0$ the valid range is $[-1, +1]$; convention picks $g = 0$ — already at the minimum.

SGD with L1 Regularization

Replace the L1 gradient with $\text{sign}(\theta)$ and run SGD as usual:

$$\theta_{t+1} = \theta_t - \alpha \left(\nabla \ell_{i_t}(\theta_t) + \lambda \text{sign}(\theta_t) \right)$$

SGD with L1 Regularization

Replace the L1 gradient with $\text{sign}(\theta)$ and run SGD as usual:

$$\theta_{t+1} = \theta_t - \alpha \left(\nabla \ell_{i_t}(\theta_t) + \lambda \text{sign}(\theta_t) \right)$$

Each step nudges every weight toward zero by $\alpha\lambda$; if a weight would cross zero, it snaps to zero and stays there.

Implementation

Step 1: Data

Generate toy data: $y = 2x + 1 + \varepsilon$

```
import torch
```

```
torch.manual_seed(0)
```

```
X = torch.randn(200, 1)           # 200 samples, 1 feature
```

```
y = 2 * X + 1 + 0.3 * torch.randn(200, 1) # true w=2, b=1
```

- X shape: (200,1) — column vector of inputs
- y shape: (200,1) — column vector of targets
- Noise $\varepsilon \sim \mathcal{N}(0, 0.09)$ added to make it realistic

Step 2: Parameters and Optimizer

```
# learnable parameters (requires_grad tells PyTorch to track them)  
w = torch.tensor(0.0, requires_grad=True)  
b = torch.tensor(0.0, requires_grad=True)  
  
optimizer = torch.optim.SGD([w, b], lr=0.1)
```

- `requires_grad=True`: PyTorch will compute gradients for these
- Optimizer receives the parameter list directly

Step 3: Training Loop

```
from torch.utils.data import TensorDataset, DataLoader

loader = DataLoader(TensorDataset(X, y),
                    batch_size=32, shuffle=True)

for epoch in range(100):
    for X_b, y_b in loader:
        y_hat = w * X_b + b           # forward pass
        loss = ((y_hat - y_b) ** 2).mean() # MSE

        optimizer.zero_grad()         # clear old gradients
        loss.backward()                # compute dL/dw, dL/db
        optimizer.step()               # w -= lr * w.grad
```

One epoch = $\lceil 200/32 \rceil = 7$ mini-batch steps.

Step 4: Inspect the Result

```
print(f"Learned: y = {w.item():.3f} * x + {b.item():.3f}")  
# Learned: y = 1.997 * x + 0.998  
# True:      y = 2.000 * x + 1.000
```

Four lines that matter in every PyTorch training loop:

1. `w * X_b + b` — forward pass (your model equation)
2. `loss.backward()` — compute $\nabla_{\theta}\mathcal{L}$
3. `optimizer.zero_grad()` — clear accumulated gradients
4. `optimizer.step()` — $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$

Full GD

1. For each epoch:
2. Compute gradient on *all* data
3. Update θ

Mini-batch SGD

1. For each epoch:
2. Shuffle data
3. For each mini-batch $\mathcal{B}$:
4. Compute gradient on $\mathcal{B}$
5. Update θ

Full GD

1. For each epoch:
2. Compute gradient on *all* data
3. Update θ

Mini-batch SGD

1. For each epoch:
2. Shuffle data
3. For each mini-batch $\mathcal{B}$:
4. Compute gradient on $\mathcal{B}$
5. Update θ

With n samples and batch size B , one epoch performs $\lceil n/B \rceil$ gradient steps instead of one.